

Индексы. Производительность запросов.

Лекция 8

Профили нагрузок

- Преимущественно чтение
- Преимущественно запись
- Смешанный

Большинство оптимизирующих техник подходят под конкретный профиль нагрузки

Общее повышение производительности СУБД

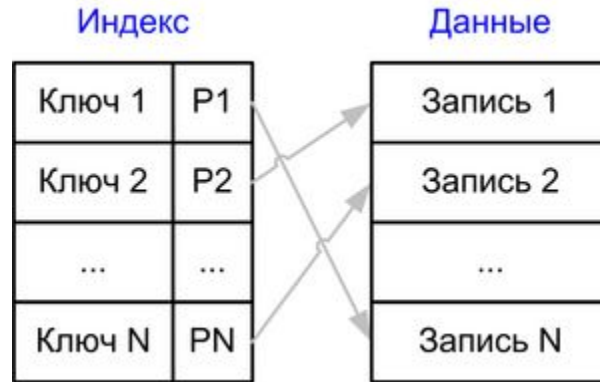
- Вертикальное масштабирование
 - Использование более быстрых накопителей и памяти
 - Использование большего количества ядер ЦП
- Горизонтальное масштабирование
 - Шардирование (распределение нагрузки по нескольким узлам)
- Использование пула соединений
- Улучшение алгоритмов работы планировщика запросов
 - Иногда планировщик “неправ”, можно помочь ему
 - Отслеживание состояния СУБД, профилировка запросов

Оптимизация нагрузки на чтение

- **Использование индексов**
- **Денормализация**
 - Храним копию данных для более быстрого доступа (например, дублируем количество лайков отдельным полем)
 - Использование материализованных представлений
- **Буфферизация**
 - Помещаем часто используемые страницы в оперативную память
- **Реплицирование (храним копии данных на разных узлах)**
 - Распределяем нагрузку по нескольким узлам

Индексы в СУБД

Индекс – это специальная структура данных, ускоряющая операции чтения.
Как правило замедляет запись, требует место на диске и в памяти.



Чтение данных в СУБД

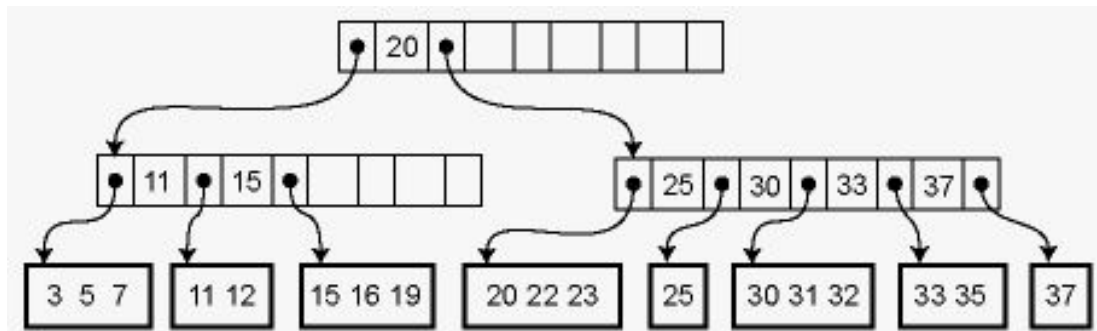
Всего есть два способа найти нужные данные:

- Полный просмотр таблицы
 - Последовательный перебор записей
 - Быстро работает на маленьких таблицах, но медленно на средних и больших
 - Если выбираем большую часть данных, то работает быстро. Иначе - медленно
- Индекс
 - Произвольный набор столбцов
 - Требуется предварительная обработка таблицы как при построении, так и при обновлении
 - Быстрый поиск в индексе, сразу получаем указатель на запись

Типы индексов в Postgres

- B-tree
- Hash
- GiST
- Sp-GiST
- GIN
- BRIN

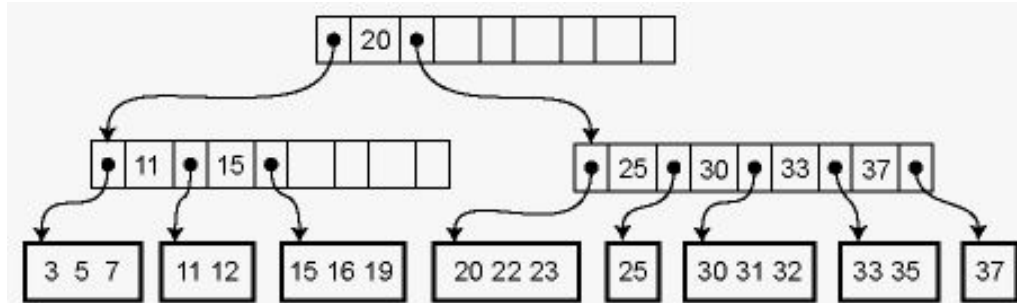
B-tree индекс (=, <, >, <=, >=)



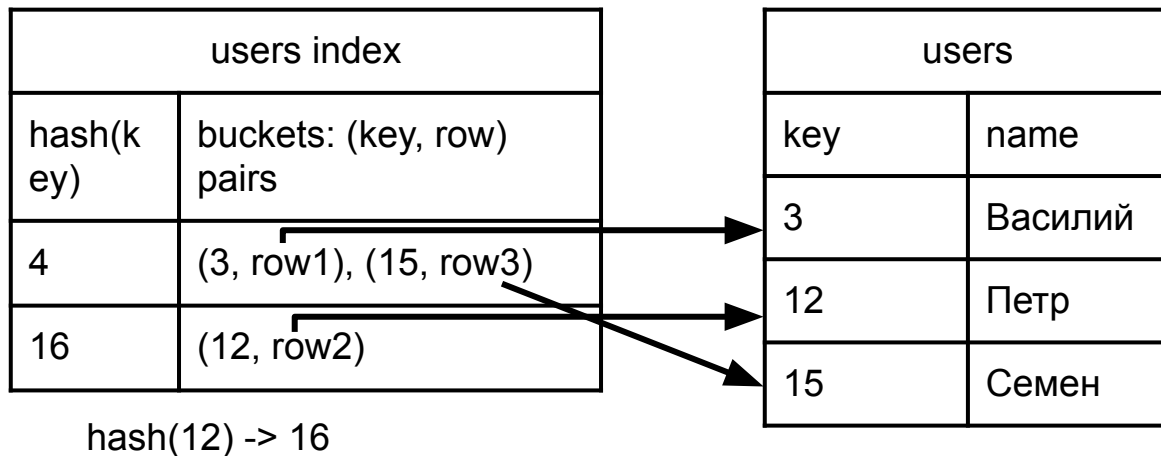
- Дерево поиска, каждый узел которого хранится в блоке (странице) фиксированного размера
- Страницы-листы хранят ссылки на страницы с данными или сами данные
- Страницы-узлы (внутренние страницы) хранят указатели на другие страницы и опорные значения ключей. Указатели и значения ключей чередуются
- Указатель слева от опорного ключа k ведет на страницу с поддеревом, листы которого хранят ключи $< k$. Правый указатель ссылается на страницу с поддеревом для ключей $\geq k$.

Свойства B-tree

- Сбалансировано
- Ключи в узлах отсортированы
- Асимптотическая сложность $O(\log(n))$
- Если у узла n детей, то сам узел содержит $n - 1$ ключей
- Каждый узел по крайней мере наполовину заполнен (кроме корня)
- Корень имеет по крайней мере 2 дочерних узла
- Коэффициент ветвления > 100 (т.е. дерево “плоское”)



Hash индекс (=)



- может использоваться для типа, у которого не определен линейный порядок
- hash-индекс может быть быстрее B-tree, т.к. “спуск” по дереву более затратная операция
- редко используется на практике

Индекс GiST (Generalized Search Tree)

Используется для:

- встроенных геометрических типов
- геометрических типов расширения PostGIS
- диапазонов
- полнотекстового поиска

Это не просто индекс, а целая инфраструктура для создания стратегий индексирования.

GiST: классы операторов

После реализации стратегии индексирования для типа могут быть доступны следующие классы операторов

<< -- левее

>> -- правее

&< -- не правее

&> -- не левее

<<| -- ниже

|>> -- выше

&& -- пересекается

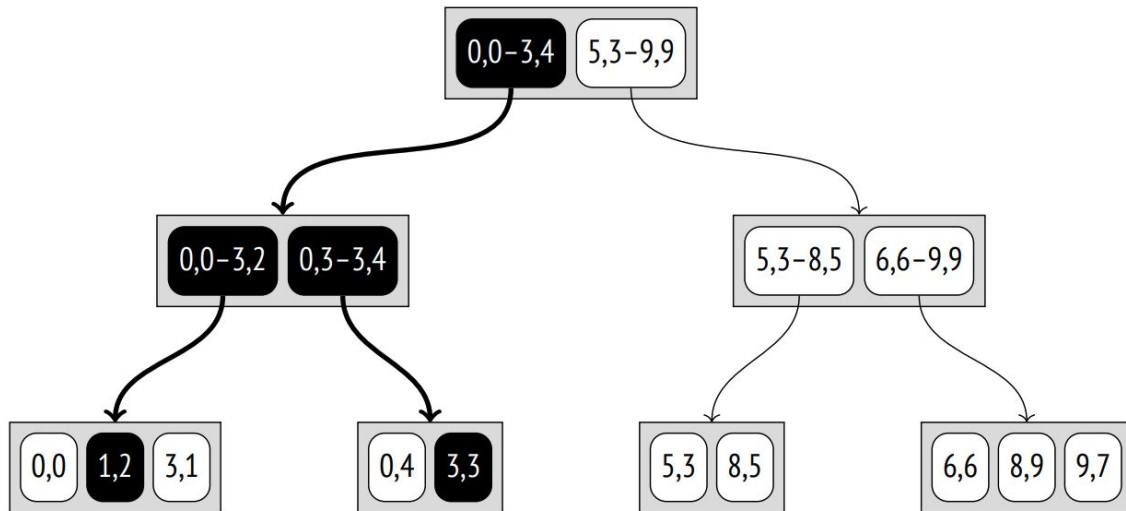
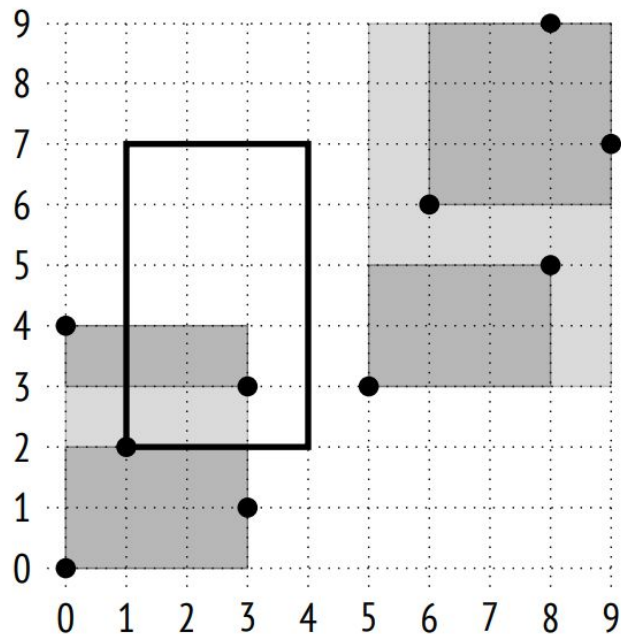
<-> -- расстояние

@> -- содержит

<@ -- содержится в

~= -- совпадает

GiST: пример R-дерево для поиска вхождения точек в прямоугольник



- По аналогии с B-tree искомые значения хранятся в листовых узлах
- Вместо сравнения используется функции согласованности: проверяет пересечение листа с искомой областью

SP-GiST (Space-Partitioned GiS)

Для неравномерно распределенных данных:

- IP адрес
- Телефонные номера
- Доменные имена
- Префиксный поиск

Принцип схож с GiST, но листьях деревьев поиска всегда непересекающиеся данные.

GiST: деревья сбалансированные и плоские

SP-GiST: деревья несбалансированные и слабо ветвятся (глубокие)

GIN (Generalized Inverted Index) индекс

Применяется для:

- полнотекстового поиска
- поиска элементов в документе JSONB
- поиска элементов в массиве

Принцип работы inverted индекса

documents	
id	text
1	The quick brown fox
2	The lazy dog jumps
3	A quick brown fox jumps

index	
token	ids
quick	[1, 3]
dog	[2]
jumps	[2, 3]

- Текст разбивается на токены (термы). Токенам ставится в соответствие id документа.
- Индекс хранит соответствие токенов и списков id, где встречается данный токен
- Индекс можно расширить позициями токена внутри документа (позволяет искать фразы)

BRIN (Block Range INdexes) индекс

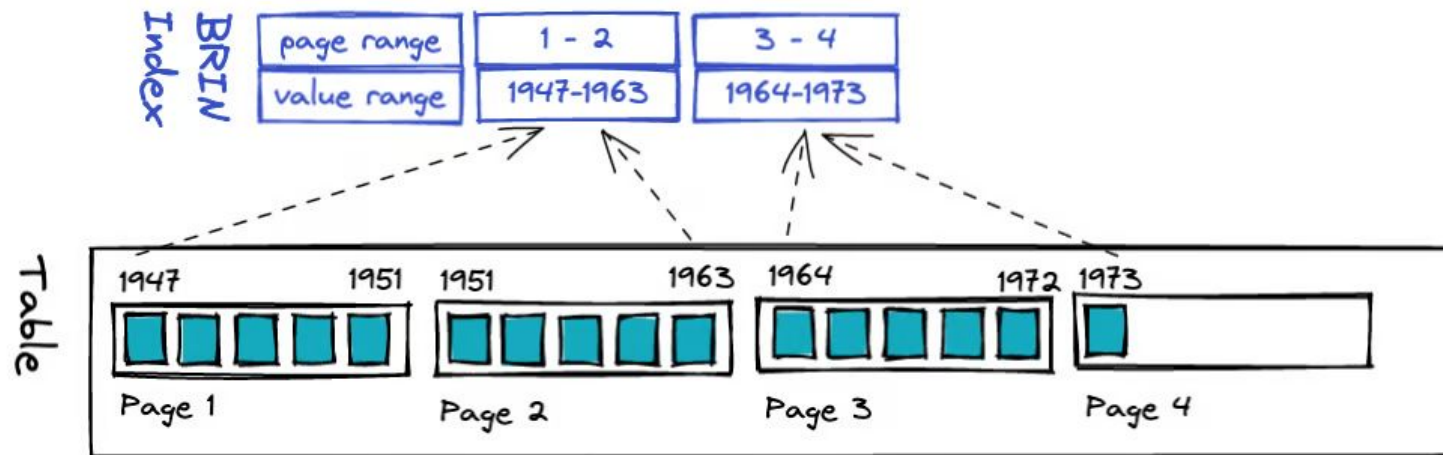
BRIN-индексы хранят обобщенные сведения о значениях, находящихся в физически последовательно расположенных блоках таблицы.

Применяется для огромных таблиц (сотни гигабайт) и для данных, которые физически отсортированы (timestamp, автоинкремент).

Примеры использования:

- таблица с логами
- таблица с трэкингом данных GPS
- таблица измерений IoT девайса

Принцип работы BRIN индекса



BRIN индекс не хранит указатели на конкретные записи, а оперирует диапазонами данных. Благодаря этому размер BRIN индекса может быть в 100 - 1000 раз меньше, чем B-TREE

Создание и удаление индексов

Базовый синтаксис:

```
CREATE INDEX index_name ON table_name(column_name);
```

Типы индексов:

```
CREATE INDEX ... USING btree      -- по умолчанию (не указывать)
CREATE INDEX ... USING hash       -- только для =
CREATE INDEX ... USING gin        -- массивы, JSONB, fulltext
CREATE INDEX ... USING gist       -- геометрия, fulltext
CREATE INDEX ... USING brin       -- большие отсортированные данные
```

Удаление:

```
DROP INDEX index_name;
```

Создание индекса: примеры

Ускорим поиск по фамилии читателя:

```
CREATE INDEX idx_readers_last_name ON readers(last_name);
```

```
SELECT * FROM readers WHERE last_name = 'Иванов';
```

 – ускоряет такие запросы

Ускорим joins и поиск по внешнему ключу:

```
CREATE INDEX idx_book_loans_reader_id ON book_loans(reader_id);
```

```
CREATE INDEX idx_book_loans_book_id ON book_loans(book_id);
```

```
SELECT b.title, r.first_name, r.last_name FROM book_loans bl
    JOIN readers r ON r.reader_id = bl.reader_id
    JOIN books b ON b.book_id = bl.book_id;
```

 – ускоряет такие запросы

Создание индекса: примеры

При добавлении колонки с ограничением целостности **UNIQUE** индекс добавляется автоматически, но можем создать его отдельно:

CREATE UNIQUE INDEX idx_books_isbn **ON** books (**isbn**) ; – гарантирует уникальность и ускоряет поиск

Hash индекс для проверки на равенство:

CREATE INDEX idx_books_isbn_hash **ON** books **USING** HASH(**isbn**) ;

BRIN для больших последовательных данных:

CREATE INDEX idx_book_loans_date_brin **ON** book_loans **USING** BRIN(**loan_date**) ;

Индексы на выражениях

Индекс для регистронезависимого поиска:

```
CREATE INDEX idx_readers_last_name_lower ON readers(LOWER(last_name));
```

```
SELECT * FROM readers WHERE LOWER(last_name) = 'иванов';
```

 – теперь быстро

Индекс на процент доступных книг:

```
CREATE INDEX idx_books_availability_percent  
ON books((available_quantity * 100 / NULLIF(quantity, 0)));
```

```
SELECT * FROM books
```

```
WHERE (available_quantity * 100 / NULLIF(quantity, 0)) < 50;
```

 – ускорили

Индексы для LIKE

Индекс для LIKE 'prefix%':

```
CREATE INDEX idx_books_title_pattern ON books(title text_pattern_ops);
```

Ускорили:

```
SELECT * FROM books WHERE title LIKE 'Война%';
```

Что если нужно ускорить LIKE '%suffix':

```
CREATE INDEX idx_books_title_reverse  
ON books(reverse(title) text_pattern_ops);
```

Ускорили:

```
SELECT * FROM books WHERE reverse(title) LIKE reverse('%мир');
```

Полнотекстовый поиск

`to_tsvector` – функция, конвертирующая текст в `tsvector` – структуру, представляющая собой словарь лексем и их позицию в документе

```
SELECT to_tsvector('english', 'The quick brown fox jumps over the lazy dog');
```

```
to_tsvector ↴
```

```
'brown':3 'dog':9 'fox':4 'jump':5 'lazy':8 'quick':2
```

```
CREATE INDEX idx_books_title_fts  
ON books USING GIN(to_tsvector('russian', title)); – создали индекс
```

Теперь можем искать:

```
SELECT title FROM books  
WHERE to_tsvector('russian', title) @@ to_tsquery('russian', 'война');
```


Индексируем массив

```
CREATE INDEX idx_book_tags_genres ON book_tags USING GIN(genres);
```

Ускорили:

```
SELECT * FROM book_tags WHERE genres @> ARRAY['Роман'];
```

```
SELECT * FROM book_tags WHERE 'Фэнтези' = ANY(genres);
```

Какие еще бывают индексы?

- Составные
- Покрывающие
- Частичные

Составные индексы

```
CREATE INDEX idx_books_publisher_year ON books(publisher, publication_year);
```

Ускорит:

```
SELECT * FROM books WHERE publisher = 'ACT' AND publication_year = 2020;
```

Также ускорит поиск по первому столбцу из индекса:

```
SELECT * FROM books WHERE publisher = 'ACT';
```

Но не ускорит:

```
SELECT * FROM books WHERE publication_year = 2020;
```

Порядок полей в индексе важен!

Покрывающий индекс

Это индекс, в котором также хранятся *неключевые* столбцы, которые не могут использоваться в условии поиска.

```
CREATE INDEX idx_books_publisher_include ON books(publisher)
INCLUDE (title, pages, publication_year);
```

Такой запрос может быть выполнен без обращения к самой таблице, используя только индекс (Index-Only Scan):

```
SELECT title, pages, publication_year
FROM books WHERE publisher = 'ACT';
```

Всегда ли индекс полезен?

Селективность (cardinality)

users			
...	phone	city	gender
	88005553535	Саратов	m
	88008005555	Казань	f
	81231231231	Пермь	m

Высокая
селективность, ???, индекс ??
индекс
эффективен

Селективность
индекс ??

Низкая
селективность,
индекс не
работает

$$\text{Селективность} = \frac{\text{кол-во уникальных значений}}{\text{общее кол-во значений}}$$

$$0 < \text{селективность} \leq 1$$

Макс. селективность 1, когда кол-во уникальных значений = общему кол-ву значений

Частичный индекс

```
CREATE INDEX idx_readers_active_email  
ON readers(email) WHERE is_active = true; – создали индекс только для активных читателей
```

```
SELECT * FROM readers  
WHERE email = 'ivanov@mail.ru' AND is_active = true; – ускорили
```

Частичный индекс позволяет:

- экономить место на диске
- убрать “популярные” значения из индекса, по которым индексирование бесполезно

Index maintenance

Посмотреть все индексы таблицы:

```
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'books';
```

Посмотреть статистику по индексам:

```
SELECT indexrelname,  
       pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,  
       idx_scan AS index_scans,  
       idx_tup_read AS tuples_read,  
       idx_tup_fetch AS tuples_fetched  
FROM pg_stat_user_indexes  
ORDER BY pg_relation_size(indexrelid) DESC;
```


Анализ производительности

Методы доступа

Характеризует способ просмотра таблиц и извлечения нужных строк.

- **последовательный просмотр** (sequential scan) – не происходит обращения к индексам, строки извлекаются из страниц таблиц. Применяется, когда селективность выборки низка (много строк)
- **просмотр на основе индекса** (index scan) – использует индекс для отыскания нужного ключа, а затем извлекает строку по идентификатору
- **просмотр исключительно на основе индекса** (index only scan) – не происходит обращения к таблице (все данные в индексе), но нужна visibility map, где показаны страницы, которые видны всем транзакциям
- **просмотр на основе битовой карты** (bitmap scan) – ищем в индексе для формирования битовой карты, которая показывает страницы для нужных строк, затем строки извлекаются из страниц, но обращение к странице происходит *только один раз*

Способы соединения наборов строк (join)

- **вложенный цикл** (nested loop) – перебираются строки из внешнего набора, для каждой ищется строка во внутреннем. Использует индекс. Используется для малых выборок.
- **соединение с хэшированием** (hash join) – строки одного набора (меньшего) помещаются в хэш-таблицу, а строки второго набора перебираются с поиском по хэш таблице. Только при соединениях с равенством. Применяется в отсутствии индексов и для больших выборок. Важно достаточное количество work_mem.
- **соединение слиянием** (merge join) – аналогично сортировке слияние, наборы должны быть отсортированы. Только при соединениях с равенством.

План запроса

Для анализа плана запроса используется команда EXPLAIN

```
EXPLAIN SELECT * FROM books b;
```

□ QUERY PLAN ▾

Seq Scan on books b (cost=0.00..1.27 rows=27 width=1024)

Оценка ресурсов
для начала вывода

Оценка общей
стоимости

Оценка кол-ва
строк

Оценка среднего
размера строк

План запроса: простые примеры

```
EXPLAIN SELECT * FROM books WHERE book_id = 1;
```

□ QUERY PLAN ▾

Seq Scan on books (cost=0.00..1.34 rows=1 width=1024)

Filter: (book_id = 1)

```
EXPLAIN SELECT * FROM books ORDER BY book_id;
```

□ QUERY PLAN ▾

Sort (cost=1.91..1.98 rows=27 width=1024)

Sort Key: book_id

-> Seq Scan on books (cost=0.00..1.27 rows=27 width=1024)

План запроса: пример с join

```
EXPLAIN SELECT b.title, bl.loan_date FROM books b
JOIN book_loans bl ON b.book_id = bl.book_id
WHERE b.book_id = 1;
```

QUERY PLAN

Nested Loop (cost=0.00..2.65 rows=1 width=520)

-> Seq Scan on books b (cost=0.00..1.34 rows=1 width=520)

Filter: (book_id = 1)

-> Seq Scan on book_loans bl (cost=0.00..1.30 rows=1 width=8)

Filter: (book_id = 1)

Выполнение запроса и анализ

```
EXPLAIN ANALYSE SELECT * FROM books b;
```

```
EXPLAIN (ANALYSE, COSTS OFF) SELECT * FROM books b;
```

☐ QUERY PLAN 

Seq Scan on books b (actual time=0.139..0.144 rows=27 loops=1)

Planning Time: 0.102 ms

Execution Time: 0.164 ms

actual time измеряется в миллисекундах

Как оптимизировать запросы?

Ищем проблемы:

- Seq Scan вместо Index Scan
- Высокая стоимость (cost)
- Много времени (actual time)

Решения:

- добавить индексы на WHERE
- добавить индексы на FOREIGN KEY для JOIN
- увеличить work_mem для Hash Join
- обновить статистику: `ANALYZE table_name;`

Когда обновляется статистика?

При “нормальном” режиме работы статистика обновляется автоматически вместе с запуском AUTOVACUUM.

Важные параметры:

- `autovacuum_analyze_threshold` – минимальное число добавленных, изменённых или удалённых кортежей, при котором будет выполняться ANALYZE для отдельно взятой таблицы (по-умолчанию 50)
- `autovacuum_analyze_scale_factor` – процент от размера таблицы, который будет добавляться к `autovacuum_analyze_threshold` при выборе порога срабатывания ANALYZE (по-умолчанию 10%)
- `autovacuum_naptime` – минимальная задержка между двумя запусками `autovacuum` (по умолчанию 60 секунд)

Для таблицы в 1000 строк: $1000 \times 0.1 + 50 = 150$ изменений

<https://postgrespro.ru/docs/postgresql/current/runtime-config-vacuum>

Когда обновлять статистику вручную?

- После массовых (bulk) операций вставки, обновления или удаления
- После добавления индекса на выражение
- После изменения типа данных колонки
- Когда оценка количества строк сильно отличается от фактического

Можно также запустить `VACUUM ANALYSE table_name;` ИЛИ `VACUUM ANALYSE;` для всей базы данных. Эти команды не только обновят статистику, но и очистят “мертвые” строки.

Статистика AUTOVACUUM

SELECT

schemaname,	
relname AS table_name,	
n_live_tup AS live_rows,	-- живые строки
n_dead_tup AS dead_rows,	-- мертвые строки (после UPDATE/DELETE)
n_mod_since_analyze AS modified,	-- изменено с последнего ANALYZE
last_vacuum,	-- когда был последний VACUUM
last_autovacuum,	-- когда был последний AUTOVACUUM
last_analyze,	-- когда был последний ANALYZE
last_autoanalyze	-- когда был автоматический ANALYZE

FROM pg_stat_user_tables

WHERE schemaname = 'mmp_assistant'

ORDER BY n_mod_since_analyze DESC;

<https://postgrespro.ru/docs/postgresql/18/monitoring-stats#MONITORING-PG-STAT-ALL-TABLES-VIEW>

Fillfactor

Это параметр хранения, задающий, на сколько процентов должна быть заполнена каждая страница при вставке строк в таблицу или индекс.

- 100 (по-умолчанию для таблиц) – только чтение или добавление (append-only). Максимальная экономия дискового пространства.
- 80–90 – регулярные обновления.
- ~50 – интенсивные и массовые обновления

Fillfactor < 100% позволяет выполнять HOT-update

Больше информации <https://habr.com/ru/companies/otus/articles/905882/>

REINDEX

`REINDEX INDEX my_index;` ИЛИ `REINDEX TABLE my_table;` полностью перестраивает индекс или все индексы таблицы

Применяется редко, если:

- индекс был поврежден: видим ошибки «bad page id» или «could not read block»
- индекс “раздут” из-за постоянных обновлений: проверить можем через представление `pg_stat_user_indexes` и размер индекса в сравнении с данными
- поменяли параметр индекса, например `fillfactor`

Оптимизация нагрузки на запись

- Вставка большими объемами (bulk writing)
 - `COPY`
 - `INSERT INTO t VALUES (1), (2), (3) ...;`
- Отключение индексов
- Write-Behind Caching
 - Сначала пишет в In-memory хранилище, затем группируем изменения с минимизацией запросов на запись
- Тонкая настройка СУБД – повышаем производительность в обмен на отказоустойчивость
 - `synchronous_commit=off`
 - увеличиваем `commit_delay`
 - увеличиваем `max_wal_size`

<https://stackoverflow.com/questions/12206600/how-to-speed-up-insertion-performance-in-postgresql>